NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4172 TinyML - Optimize Techniques for Tiny ML: Neural Network Design

Assoc Prof Nicholas Vun
College of Computing and Data Science

References

(i) Measuring what Really Matters: Optimizing Neural Networks for TinyML

Measuring what Really Matters: Optimizing Neural Networks for TinyML

Lennart Heim
ETH Zürich
RWTH Aachen
leheim@ethz.ch

Andreas Biri
ETH Zürich
abiri@ethz.ch

Zhongnan Qu
ETH Zürich
quz@ethz.ch

Lothar Thiele
ETH Zürich
thiele@ethz.ch

(ii) MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

Ji Lin[1] Wei-Ming Chen[1] Han Cai[1] Chuang Gan[2] Song Han[1]
[1]MIT [2]MIT-IBM Watson AI Lab
https://mcunet.mit.edu

Optimizing of Neural Network Performance

Focus of machine learning performance:

- inference latency
- accuracy

Typical approaches

- bigger datasets
- increase in complexity of NN model
- improve (more complex) CPUs and accelerators architectures
  E.g., GPUs, custom ASICs
  - aimed at boosting parallel math performance.

NN Performance for Tiny ML application

Tiny ML for microcontroller (MCU)

- performing AI at the edge
  E.g. Object detection, Keyword detection

Requirements

- Real time inference
- Low power consumption
- Low cost
- Good accuracy

Challenges – Resource constraint of MCU:

- Limited memory size
- Average computation power
- Energy efficiency

Tiny ML on MCU

MCU

- relatively simpler hardware architecture
- no co-processor
- memory resources are inherently limited
  - E.g. no cache memory

Existing deployment frameworks

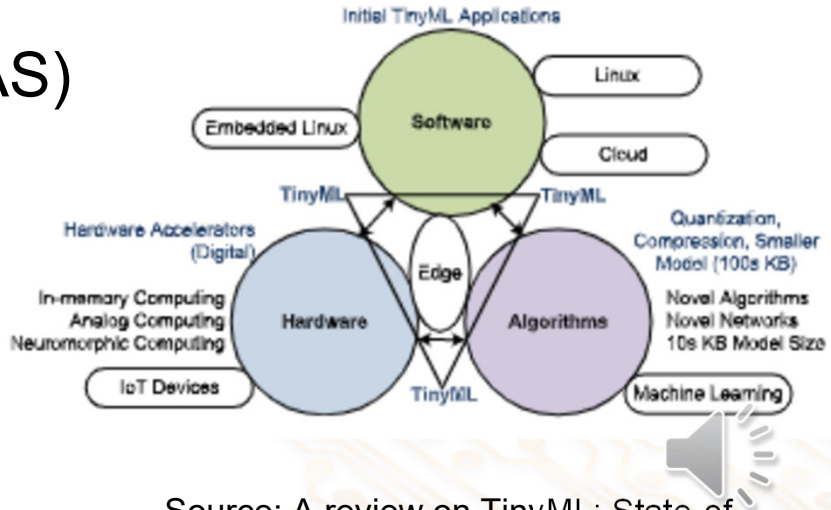
- TensorFlow Lite Micro
- CMSIS-NN
- Edge Impulse
- TinyEngine
- MicroTVM
- etc

Efficient NN for Tiny ML Inference

Some approaches (attempts) to have more efficient deep NN for MCU

- reducing the number of MAC and FLOP operations
  - pruning
- compressing NNs for efficient on-device inference
  - quantization
- exploring more efficient models
  - using neural architecture search (NAS)

- Efficient embedded design techniques

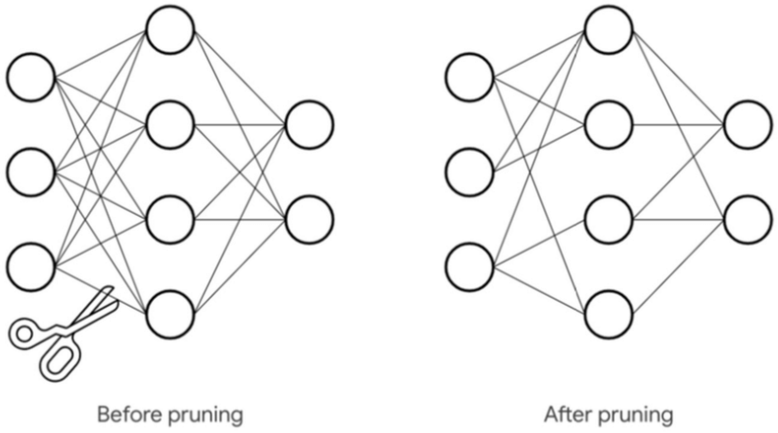


Source: A review on TinyML: State-of-the-art and prospects by Partha Pratim Ray

Pruning Trained model

Pruning

- deleting low value parameters from an existing neural network
- reduce the amount of computing necessary to run the neural network.
  - enhancing its efficiency while retaining the network's accuracy



Source: analyticsindiamag.com/a-beginners-guide-to-neural-network-pruning/

Compressing Trained Model by Quantization

Reducing the bit-width of NN parameters - Quantization

- permits a drastic reduction of the memory footprint

Has become a standard compression technique in TinyML

- significant memory savings while usually having a negligible effect on accuracy

Most common quantization

- 32 bits floating point to 8 bits integer

If used on a 32-bit hardware architecture

- acceleration of up to 4× is expected
  - if aligned memory access and SIMD can be leveraged.

But limited number of registers in MCUs and the NN structure can limit this exploitation

Quantization Approaches

Two Quantization algorithms

- post-training quantization (PTQ)
- quantization-aware training (QAT)

PTQ quantizes an already-trained float (TensorFlow) model

- convert the parameters to 8-bit integers
  - E.g., TensorFlow Lite format using the TensorFlow Lite Converter
- easier to implement

QAT re-trains a model with quantized parameters

- until it converges to a point with a better loss
- better model accuracy

Weight Regularization (During Training)

Weight Regularization put constraints on the complexity of a model by forcing its weights to take only small values

- makes the distribution of weight values more *regular*
- simpler model is less prone to overfitting (compared to a complex one)

Done by adding to the loss function of the model a cost associated with having large weights:

i. *L1 regularization*—The cost added is proportional to the *absolute value of the weight coefficients* (the *L1 norm* of the weights).

ii. *L2 regularization*—The cost added is proportional to the *square of the value of the weight coefficients* (the *L2 norm* of the weights).

In Keras, weight regularization is added by adding *kernel_regularizer* keyword as arguments to the layer:

```
layers.Dense(16, kernel_regularizer=l2(0.002),activation="relu"),
```

Efficient Architectures for Mobile devices

New architectures designed specifically for mobile devices (Smart phone)

- MobileNets
- SqueezeNet
- SparseNet
- ShuffleNet
- EfficientNet

In addition to accuracy, also consider network complexity

- intererplay between accuracy, number of parameters, activations, and operations.

But their applicability to TinyML is still rather limited

- resources on such systems are far more constrained than for mobile phones.

Neural Architecture Search (NAS)

Using ML to search for efficient architectures in an automated manner

- by systematically exploring the design space using a set of selected evaluation criteria.

NAS system can be split into three components

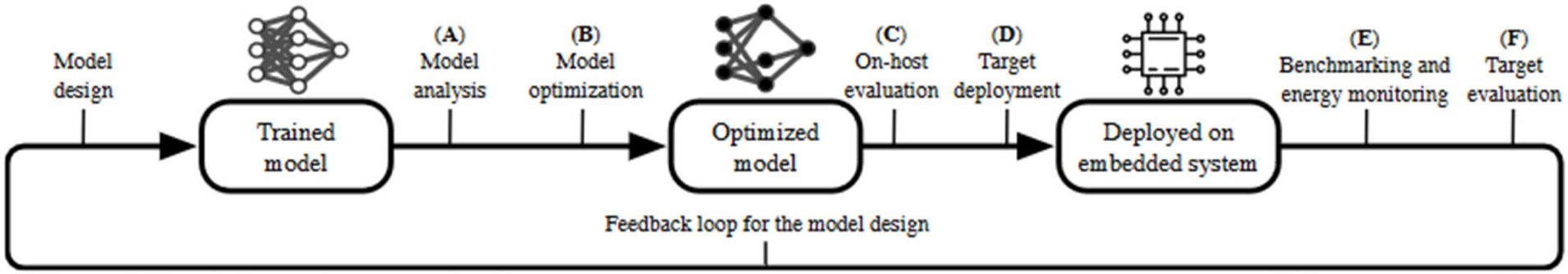
- search space: type of architecture that can be used by the algorithm (has to be redesigned and reduced due to limited memory in TinyML)
- search algorithm: how to experiment with different neural networks
- evaluation strategy: specific to application requirements and hardware limitation of MCU, E.g. latency and energy cost constraint

Example of a NAS based DNN: EfficientNet

Metric for NAS search and training time is given in GPU-days

- expensive and time consuming
- can be thousands of GPU-days

Flowchart to optimize NN for TinyML



Source: Measuring what Really Matters: Optimizing Neural Networks for TinyML

Target deployment

Use specialized optimizations that are available for the target device

- software acceleration library like CMSIS-NN
  - provide the possibility to leverage dedicated, hardware-optimized kernels.
- increase the execution efficiency by leveraging target-specific features like SIMD instructions

CMSIS-NN

CMSIS: Common Microcontroller Software Interface Standard

CMSIS–NN: a collection of efficient neural network software acceleration library developed for Arm Cortex-M processor to

- maximize the performance
- minimize the memory footprint of neural networks cores
- targeted for intelligent IoT edge devices

“Converting a Neural Network for Arm Cortex-M with CMSIS-NN”
https://developer.arm.com/documentation/102591/0000

“How to Achieve High-Accuracy Keyword Spotting on Cortex-M Processors”
https://community.arm.com/arm-community-blogs

Experimental Results

	CMSIS-NN	TFL Acc.	TFLM Acc.	Δ Acc.	Memory Footprint [KiB]
U	—	98.79%	98.79%	0.00%	320.28
Q	✗	98.74%	98.74%	0.00%	85.19
	✓	98.74%	98.76%	**0.02%**	

Source: Measuring what Really Matters: Optimizing Neural Networks for TinyML

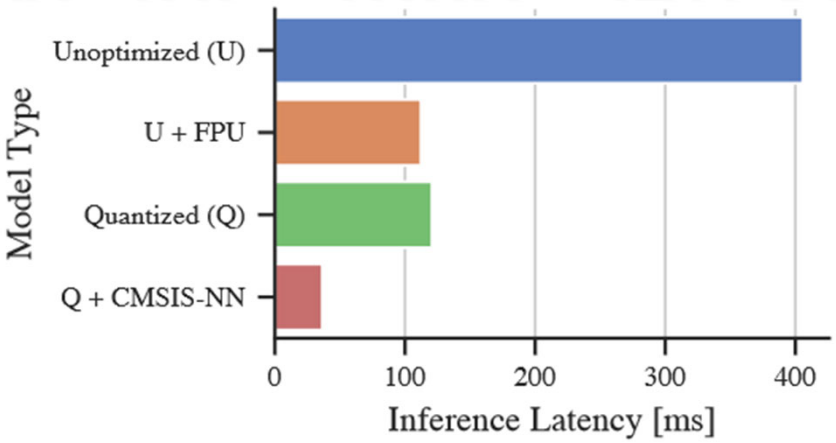
Compared to the unoptimized model (U)

- 8 bit quantized model (Q) has a decreased accuracy by 0.05 % for the LeNet.
- quantization drastically reduces the memory footprint to 26.6 % of the original size.

Including CMSIS-NN (which can only be verified on the MCU itself)

- slightly altered accuracy (+0.02 %) due to the different implementation of the underlying kernels
  - i.e. insignificant effect in terms of accuracy

Experimental Results - Latency



Source: Measuring what Really Matters: Optimizing Neural Networks for TinyML

Compared to the unoptimized model (U)

- using the FPU results in a 4× faster inference (U + FPU) for the LeNet.
- quantized model is also significantly faster

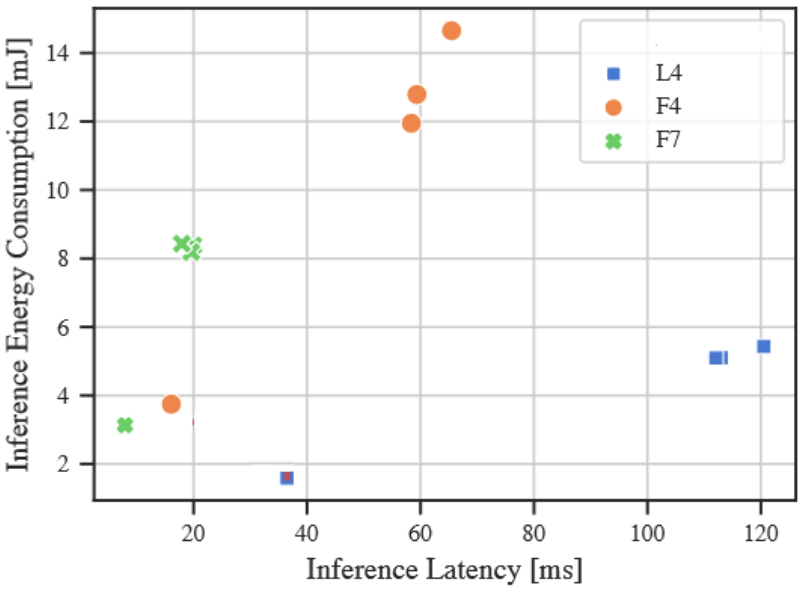
Using CMSIS-NN software acceleration library

- further accelerate fixed point operations, speeding up the quantized model by an additional 4×

Combination of quantization and CMSIS-NN

- superior in regard to memory footprint as well as latency
- without requiring the availability of an FPU

Experimental Results (cont.)



Source: Measuring what Really Matters: Optimizing Neural Networks for TinyML

Energy consumption increases with latency

- almost linear relationship between latency and energy consumption across all optimizations

Latency is due to

- memory access

Memory access

- requires magnitudes more energy than computations
  (Memory bound vs Compute bound)

NN types that lead to more memory access (e.g., dense layers)

- would require a disproportional amount of energy

Experiment Results (cont.)

	MCU	Latency [ms] U	Latency [ms] O	Speedup
LeNet	L4	470.5	36.4	**12.9×**
	F4	227.6	16.1	**14.1×**
	F7	126.4	8.1	**15.7×**
ResNet	L4	*oversized*	2217.0	–
	F4	28818.5	984.3	**29.3×**
	F7	15566.9	449.3	**34.6×**

Source: Measuring what Really Matters: Optimizing Neural Networks for TinyML

Across different models of MCU family

- optimizations always provide speedup

For bigger architectures such as the ResNet

- optimizations have a substantial effect
  - i.e., more benefit for complex network architectures

Aside: Vanishing Gradient

After the first CNN-based architecture (AlexNet) that won the ImageNet 2012 competition

- subsequent winning architecture uses more layers in a deep neural network to reduce the error rate
- but it is observed that after some depth
  - the performance degrades.

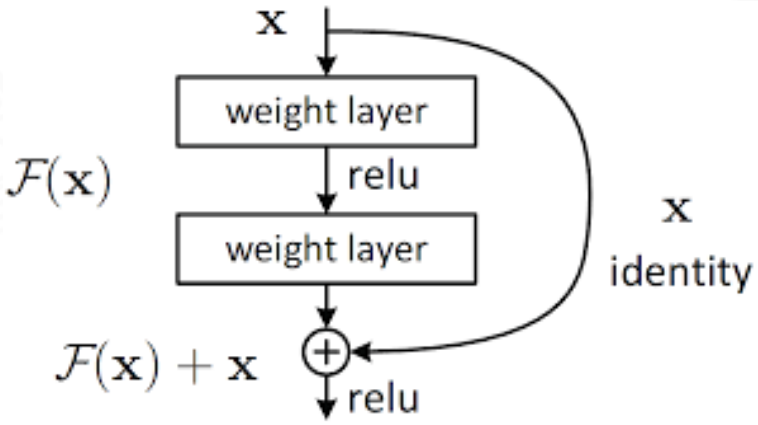
Deeper number of layers lead to “vanishing gradient”

- when the network is too deep
  - gradients used to calculate the loss function shrink to 0 after several applications of the chain rule during back propagation
  - result in weights that are never updated
  - no learning is being performed.

Aside: Resnet

Resnet (Residual Network)

- a CNN architecture designed to support hundreds (thousands?) of convolutional layers: E.g. ResNet-101
- utilize *skip connections* to jump over some layers



With ResNet

- the gradients can flow directly through the skip connections backwards from later layers to initial filters.

MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

Ji Lin[1] Wei-Ming Chen[1] Han Cai[1] Chuang Gan[2] Song Han[1]
[1]MIT [2]MIT-IBM Watson AI Lab
https://mcunet.mit.edu

Memory Bottleneck

Tiny ML learning is fundamentally different from typical deep learning due to the tight memory budget

- a common MCU usually has an SRAM smaller than 512kB
- too small for deploying most off-the-shelf deep learning networks
- limits the feature map/activation size
    - restricting us to use a small model capacity or a small input image size.

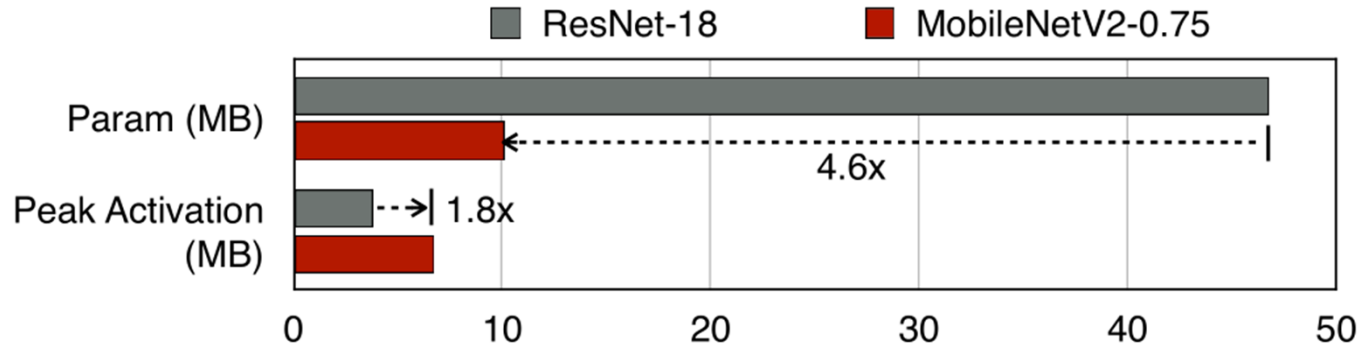
Existing work employs pruning, quantization and neural architecture search for efficient deep learning

- focus on reducing the number of parameters and FLOPs
    - but not the memory bottleneck.

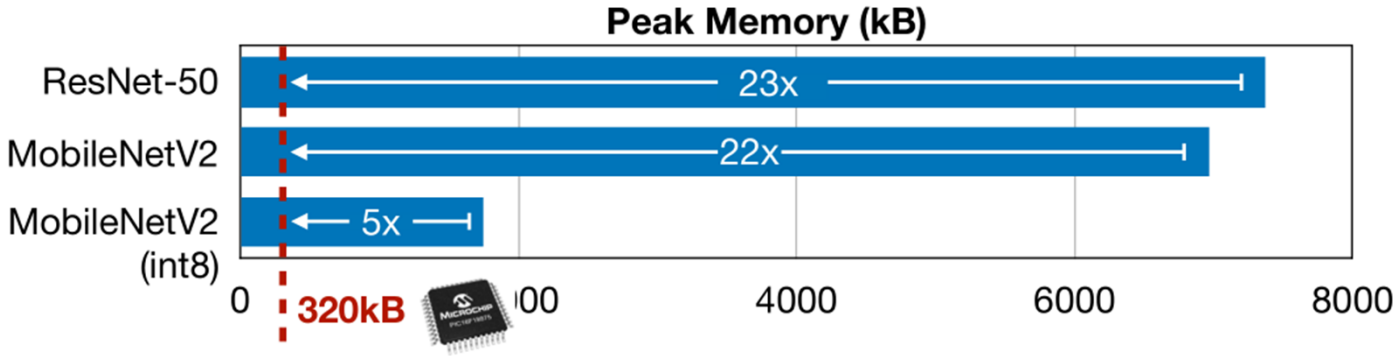
Activation Memory

Activation Memory

- the amount of memory needed during the computation



- Existing network **CANNOT** fit the tight memory constraints on MCU



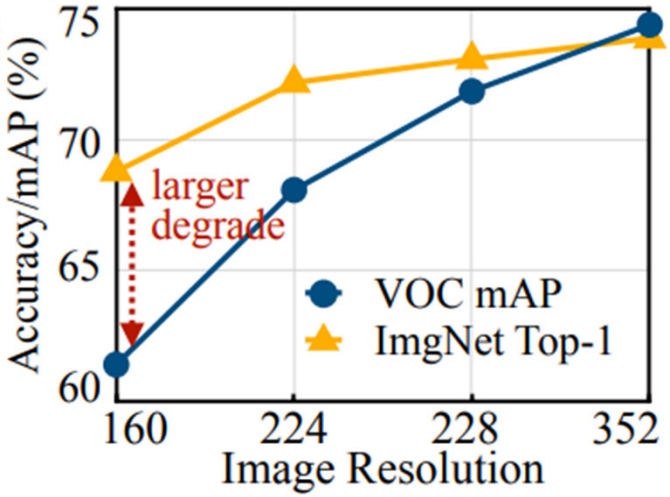
Source: https://mcunet.mit.edu/

Input resolution

Input resolutions used in TinyML

- usually small, which might be acceptable for image classification
- but not sufficient for dense prediction tasks like **object detection**

Performance of object detection degrades much faster with input resolution than image classification.



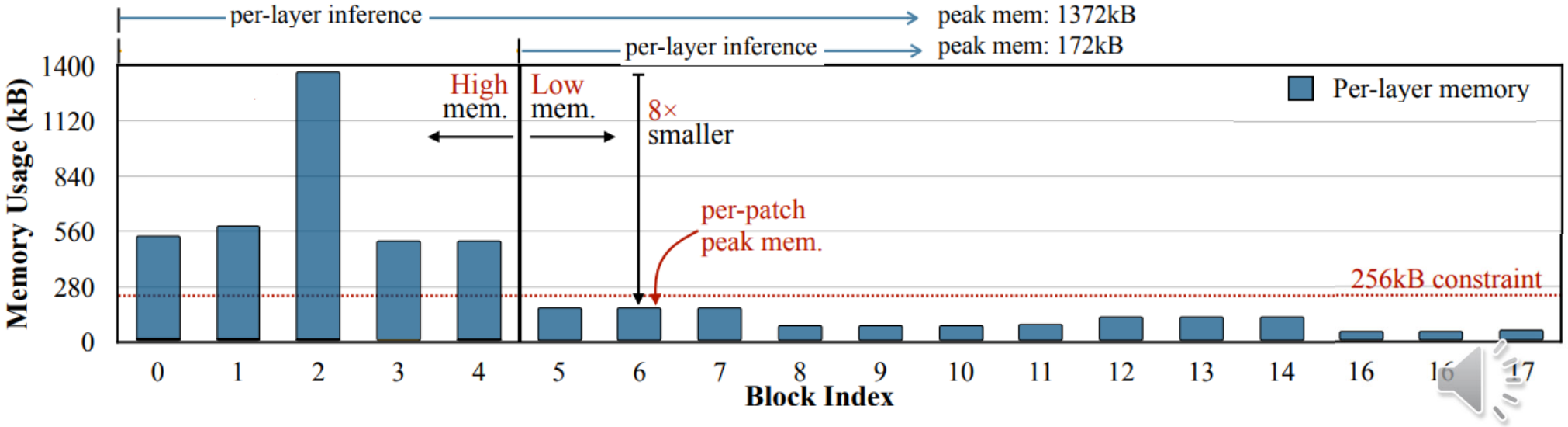
Ref: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

MobileNet

MobileNet

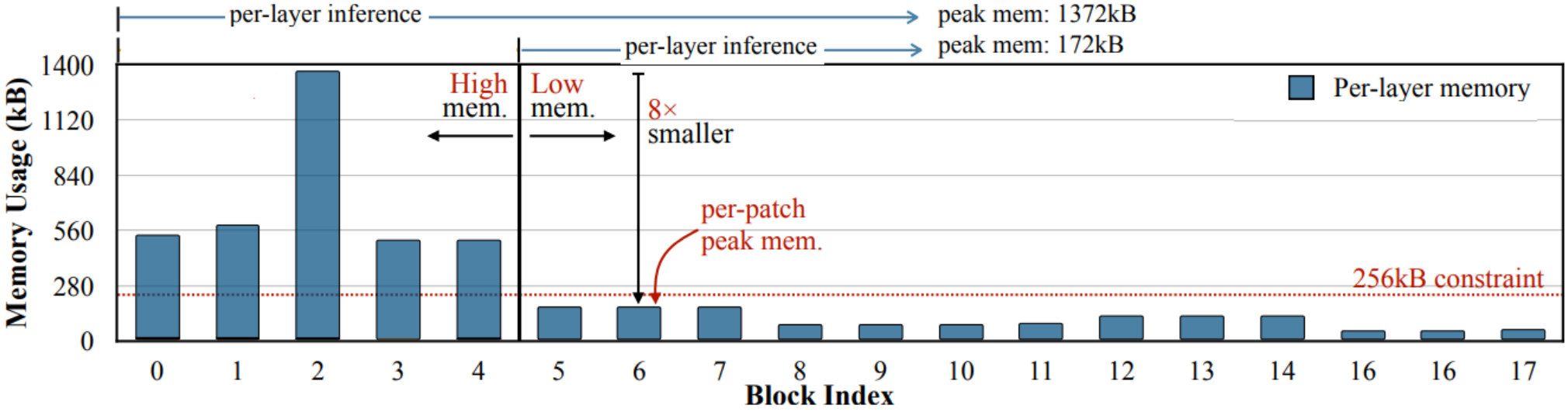
- a family of general purpose computer vision CNNs
- designed for mobile devices to support classification and detection

Very imbalanced memory usage distribution



Ref: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

MobileNetV2 Activation



Ref: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

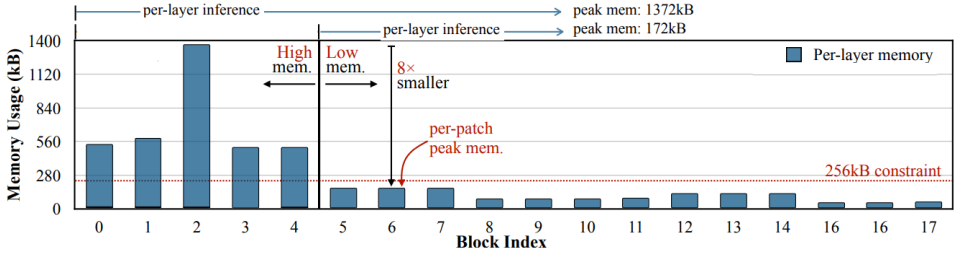
Peak memory is determined by the first 5 blocks

- while the later blocks all share a small memory usage

First 5 blocks have a high peak memory (>450kB)

- becoming the memory bottleneck of the entire network in TinyML

MobileNetV2 Activation



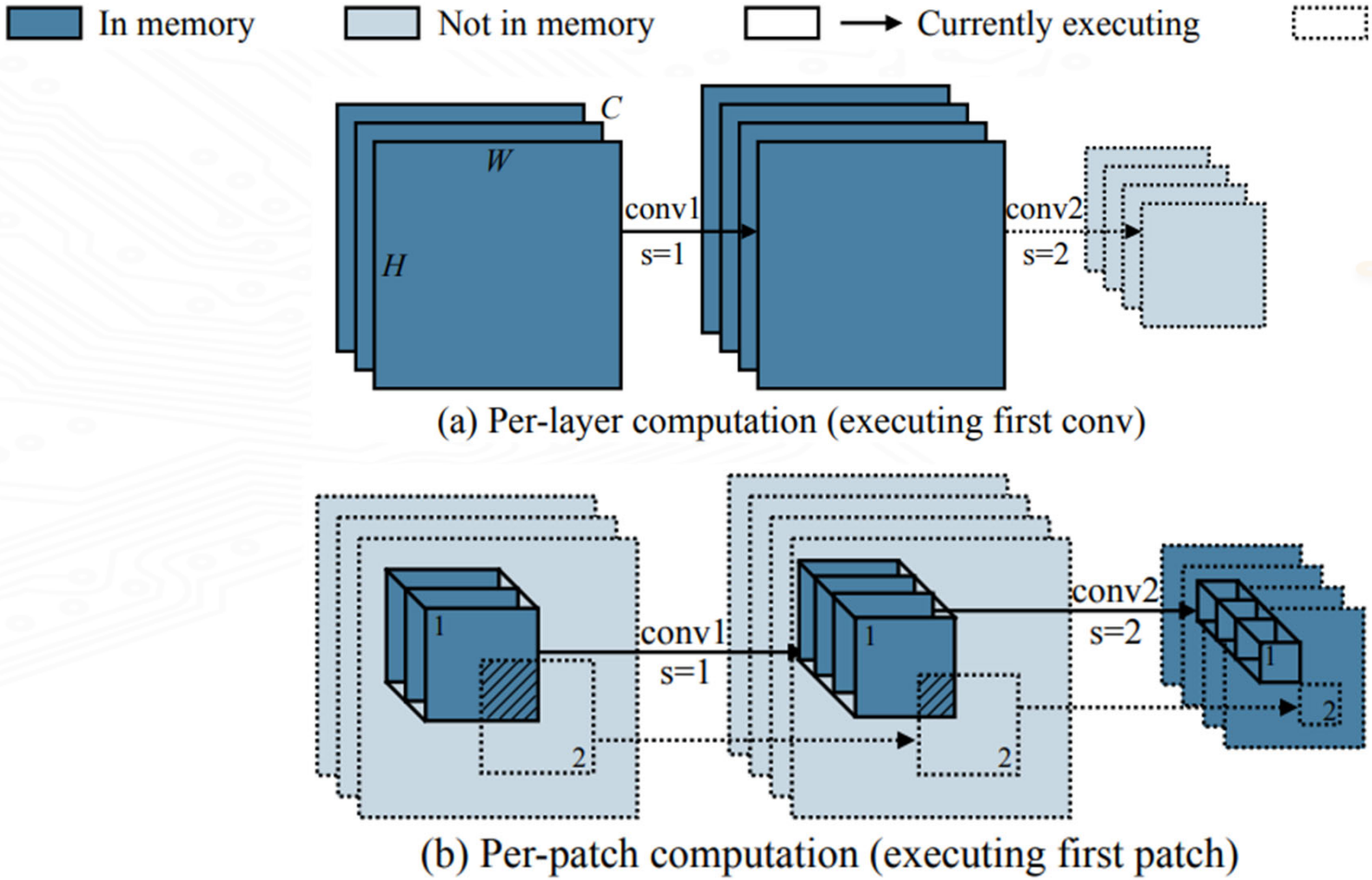
Imbalanced memory distribution

- significantly limits the model capacity and input resolution executable on MCUs.

To accommodate the initial memory intensive stage

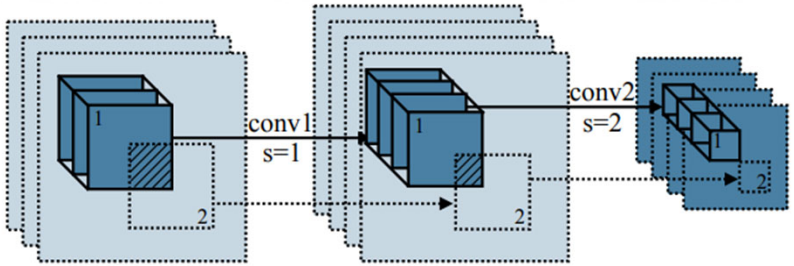
- the whole network needs to be scaled down
  - even though the majority of the network already requires a small memory usage.
- makes resolution-sensitive tasks (e.g., object detection) difficult
- high-resolution input will lead to even larger initial peak memory.

Patch-Based Inference – Layer vs Patch

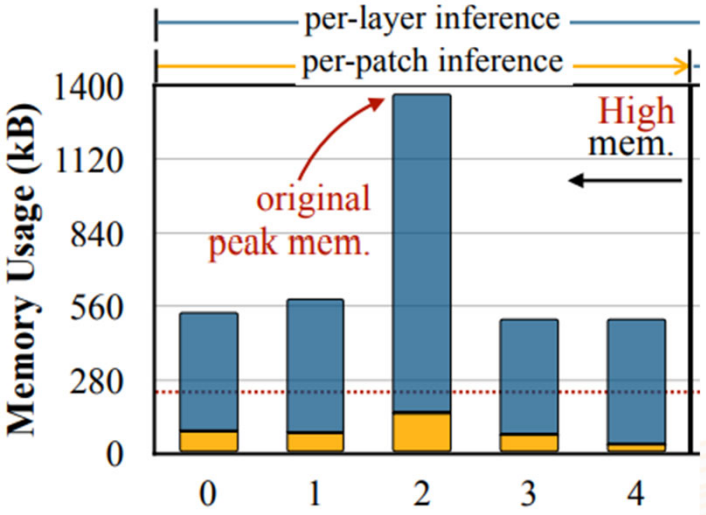


Source: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

Per-patch Inference



(b) Per-patch computation (executing first patch)



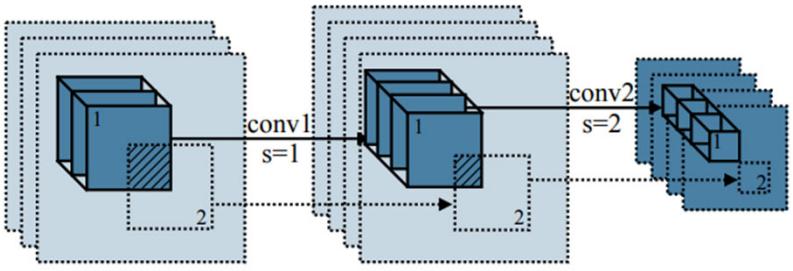
Only need to store the feature of a small patch

- can significantly cut down the peak memory of the initial stage
- can easily fit a 256kB MCU
- reduce the overall peak memory by 8×

But

- reduced peak memory comes at the price of computation overhead

Per-Patch Inference



(b) Per-patch computation (executing first patch)

Computation of the output patches

- need the input image patches to be overlapped
- lead to repeated computation

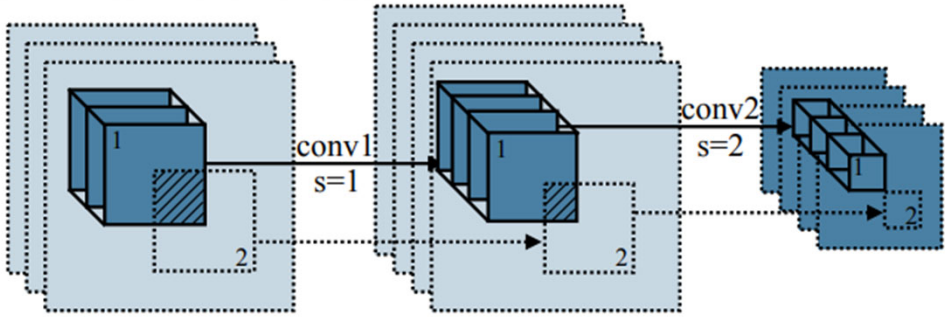
Overhead is related to the ‘receptive field’ of the initial stage

- the larger the receptive field, the larger the input patches, which leads to more overlapping.

MCUNetV2 use redistribution of the receptive field

- shift the receptive field and workload to the later stage of the network.

Per-Patch Inference



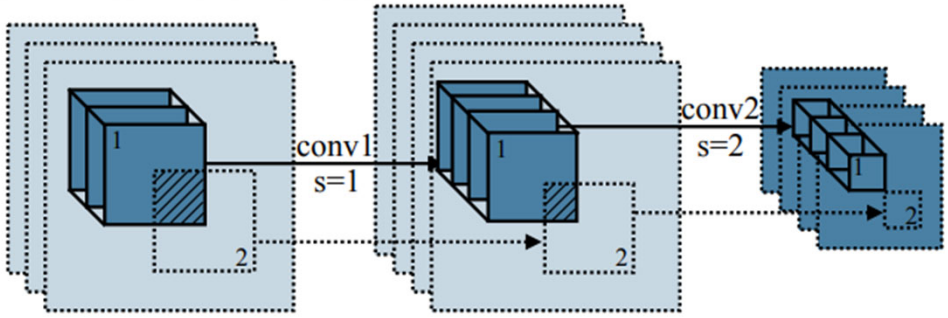
(b) Per-patch computation (executing first patch)

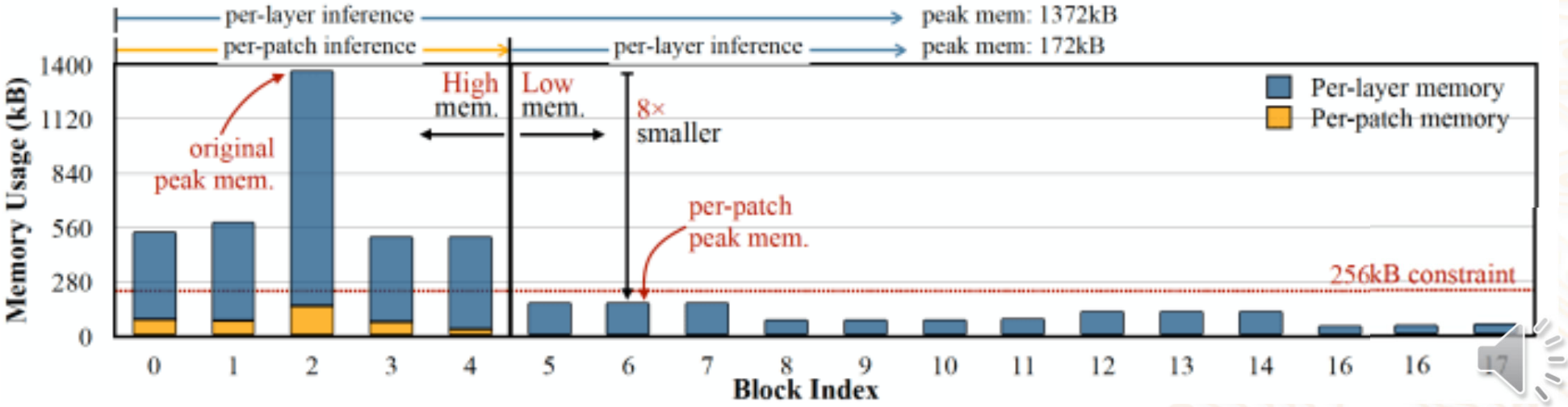
MCUNetV2's Receptive Field redistribution technique

- reduces the patch size
- reduces the computation overhead caused by overlapping
  - without hurting the performance of the network.
- more freedom trading-off input resolution, model size, etc

Source: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

Per-Patch Inference





Source: MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning

Continual Learning

Models may need to be updated to evolving data pattern

- costly to completely retrain a model from scratch

Continual learning (CL) in machine learning

- enables models to learn and update incrementally from new data collected during inference operation
- updating knowledge over time
  - but without “forgetting“ the previously learned information
  - i.e. does not overwrite existing settings

With CL, model can hence be

- ‘more accurate’ as more data become ‘available’ to update the model
- adapting to evolving environments

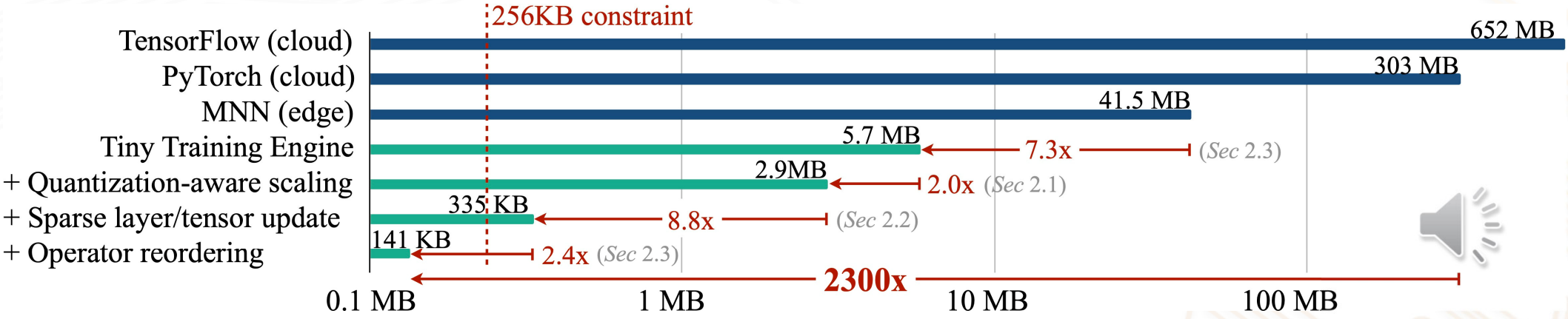
Continual Learning for TinyML

On-device training

- enables the model to adapt to new data collected from the sensors to fine-tuning a pre-trained model.
- but training memory consumption is prohibitive

MCUNetV3 (“On-Device Training Under 256KB Memory”)

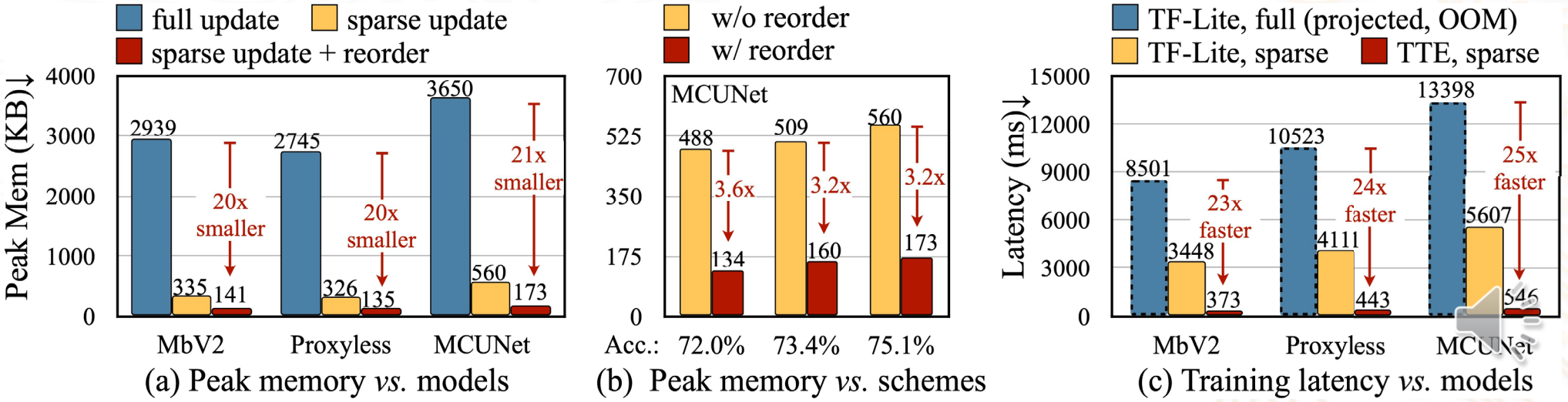
- an algorithm-system co-design framework to make on-device training with only 256KB of memory



Source: https://mcunet.mit.edu/

Tiny Training Engine

- Quantization-Aware Scaling - to calibrate the gradient scales and stabilize quantized training
- Graph optimization reordering
- Sparse Update - to skip the gradient computation of less important layers and sub-tensors



Source: https://hanlab.mit.edu/projects/mcunetv3

Federated Learning with Edge Devices

A decentralized machine learning approach using edge devices to collaboratively train an AI model

i. Central server sends a pre-trained ‘foundation’ model to edge devices
ii. Edge devices train the model using their local data
iii. Edge device return the updated model parameters (e.g., weights)
iv. Server aggregate parameters received, averaged them and integrated into a centralized updated model, which can be distributed to edge devices for inference operation

Benefits:

- Enhanced privacy: Sensitive data remains local
- Real-time insights: Models can be updated with the latest local data

Summary

TinyML is still a new and fast growing field of machine learning technologies and applications

- with lot of potentials and new opportunities
- still faces many challenges imposed by MCUs
  - Memory constraint
  - Energy constraint
  - Real-time inference latency

Many on-going studies on how to improve the performance through

- better NN model and algorithms that suits the MCU
- more effective embedded implementation techniques
- in-depth understanding of the underlying operations during the model execution